MINI PROJECT 15

Kuantisasi, Verifikasi, dan Pembekuan Paket Deployment

Mini Project ini dijalankan di komputer pengembangan. Tidak dibutuhkan hardware embedded karena integrasi fisik dan benchmark perangkat dilakukan pada MP16. Tujuan MP15 adalah menghasilkan paket deployment yang reproducible dan memilih satu model utama tanpa retraining atau post-test tuning.

Tujuan dan Handoff

Input wajib adalah project_status.json dengan status ready_for_mp15, dua model kandidat, konfigurasi feature reduction, frozen-test MP13, dan real-time contract MP14. Output wajib adalah paket INT8, C header untuk model klasik, deployment_config.json, deployment manifest, metric equivalence/latency, dan status ready_for_mp16.

Input MP14/MP13	Output MP15	Dipakai MP16 untuk
SVM + feature config	mp15_classical_model_int8.h	inference MCU
Tiny CNN float	mp15_tiny_cnn_int8_torchscript.pt	fallback/reference
realtime config	deployment_config.json	parameter integrasi
frozen-test metrics	selection policy result	model utama
ready_for_mp15	ready_for_mp16	proyek final

Tabel 15.11. Handoff Mini Project 15.

Prosedur Step by Step

1. Pastikan project_status.json bernilai ready_for_mp15 dan jangan mengubah file model MP11/MP12.

2. Pastikan environment memiliki numpy, matplotlib, scikit-learn, joblib, dan PyTorch.

3. Simpan kode lengkap di bawah sebagai mini_project_15.py pada root running project.

4. Jalankan: python mini_project_15.py.

5. Periksa terminal. Pastikan agreement SVM dan CNN terhadap model float tercetak dan status berubah menjadi ready_for_mp16.

6. Buka mp15_equivalence.png. Pastikan tidak ada perubahan label yang tidak dijelaskan.

7. Buka mp15_model_size.png dan mp15_latency.png. Ingat bahwa latency adalah host reference, bukan benchmark MCU/SBC.

8. Buka config/deployment_config.json. Catat selected_deployment_model dan pastikan post_test_tuning_performed = false.

9. Buka firmware/mp15_classical_model_int8.h dan deployment/mp15_manifest.json. Pastikan semua file yang dibutuhkan MP16 tercantum.

10. lalu jangan mengubah model atau scale sebelum memulai MP16.

Kode Python Mini Project 15

Kode berikut telah diuji end-to-end pada artefak serial Mini Project 14. Untuk Tiny CNN, kode menggunakan eager static quantization PyTorch pada host sebagai demonstrasi PTQ dan numeric-equivalence. API quantization dapat berbeda antarversi PyTorch; artefak final yang dipakai MP16 dalam running project adalah model yang tercantum pada deployment manifest.

```
from pathlib import Path
import json, time, shutil, warnings
import numpy as np
import matplotlib.pyplot as plt
import joblib
import torch
import torch.nn as nn
```

```
ROOT = Path('.')
SEED = 2026
PERF_TOL = 0.03
TARGET_FRAMES = 96
BENCH_ITERS = 120
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.set_num_threads(1)
warnings.filterwarnings('ignore', category=DeprecationWarning)


def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, indent=2), encoding='utf-8')


def load_logmel(sample_id, mu, sigma):
    d = np.load(ROOT/'features/logmel/mp09'/f'{sample_id}.npz', allow_pickle=True)
    x = d['log_mel'].astype(np.float32)
    if x.shape[0] >= TARGET_FRAMES:
        x = x[:TARGET_FRAMES]
    else:
        fill = float(np.min(x)) if x.size else 0.0
        x = np.pad(x, ((0,TARGET_FRAMES-x.shape[0]),(0,0)), constant_values=fill)
    return (x - mu) / sigma


def bench(fn, warmup=20, iters=BENCH_ITERS):
    for _ in range(warmup): fn()
    vals=[]
    for _ in range(iters):
        t0=time.perf_counter_ns(); fn(); vals.append((time.perf_counter_ns()-t0)/1e6)
    return {'mean_ms':float(np.mean(vals)), 'p95_ms':float(np.percentile(vals,95)), 'min_ms':float(np.min(vals))}

# ---------- Gate ----------
status_path = ROOT/'project_status.json'
status = json.loads(status_path.read_text(encoding='utf-8'))
if status.get('status') != 'ready_for_mp15':
    raise RuntimeError(f"Status harus ready_for_mp15, ditemukan {status.get('status')!r}")

mp13 = json.loads((ROOT/'results/metrics/mp13_final_test_metrics.json').read_text(encoding='utf-8'))
rt = json.loads((ROOT/'config/realtime_architecture_config.json').read_text(encoding='utf-8'))
red = json.loads((ROOT/'config/feature_reduction_config.json').read_text(encoding='utf-8'))
clf_cfg = json.loads((ROOT/'config/classifier_config.json').read_text(encoding='utf-8'))
dl_cfg = json.loads((ROOT/'config/deep_learning_config.json').read_text(encoding='utf-8'))
split = json.loads((ROOT/'splits/mp09_split.json').read_text(encoding='utf-8'))

if mp13.get('post_test_tuning_performed'):
    raise RuntimeError('MP13 menunjukkan post-test tuning. Deployment dihentikan.')
if rt.get('final_model_selection') != 'DEFERRED_TO_MP15':
    raise RuntimeError('MP14 tidak menyerahkan pemilihan model ke MP15.')

classes = list(clf_cfg['classes'])

# ---------- A. Linear SVM float -> INT8 reference ----------
svm = joblib.load(ROOT/clf_cfg['model_file'])
D39 = np.load(ROOT/'features/dataset/mp09_features_39d.npz', allow_pickle=True)
X39 = D39['X'].astype(np.float64)
y = D39['y'].astype(str)
train_mask = D39['train_mask'].astype(bool)
test_mask = D39['test_mask'].astype(bool)
sel = np.asarray(red['selected_indices'], dtype=np.int64)
mean39 = np.asarray(red['scaler_mean'], dtype=np.float64)
scale39 = np.asarray(red['scaler_scale'], dtype=np.float64)
Z8 = ((X39 - mean39) / scale39)[:, sel]
float_pred = svm.predict(Z8)

W = np.asarray(svm.coef_, dtype=np.float64)
b = np.asarray(svm.intercept_, dtype=np.float64)
# Input scale dan weight scale dihitung tanpa tuning test. Input calibration = training only.
train_max = float(np.max(np.abs(Z8[train_mask])))
x_scale = max(train_max / 127.0, 1e-8)
W_scales = np.maximum(np.max(np.abs(W), axis=1) / 127.0, 1e-8)
Wq = np.clip(np.round(W / W_scales[:,None]), -127, 127).astype(np.int8)


def svm_int8_scores(z):
    zq = np.clip(np.round(z / x_scale), -127, 127).astype(np.int8)
    acc = Wq.astype(np.int32) @ zq.astype(np.int32)
    return acc.astype(np.float64) * (W_scales * x_scale) + b

q_pred = np.asarray([classes[int(np.argmax(svm_int8_scores(z)))] for z in Z8])
svm_agree_all = float(np.mean(q_pred == float_pred))
svm_agree_test = float(np.mean(q_pred[test_mask] == float_pred[test_mask]))

# Export package.
model_dir = ROOT/'models'; model_dir.mkdir(parents=True, exist_ok=True)
firmware_dir = ROOT/'firmware'; firmware_dir.mkdir(parents=True, exist_ok=True)
np.savez_compressed(model_dir/'mp15_classical_int8.npz',
                    selected_indices=sel.astype(np.int32),
                    selected_mean=mean39[sel].astype(np.float32),
                    selected_scale=scale39[sel].astype(np.float32),
                    input_quant_scale=np.asarray([x_scale], dtype=np.float32),
                    weight_q=Wq,
                    weight_scales=W_scales.astype(np.float32),
                    bias=b.astype(np.float32),
                    classes=np.asarray(classes, dtype=object))

# C header for MCU reference implementation.
def c_arr(name, arr, ctype, fmt):
```

```
    flat=np.asarray(arr).reshape(-1)
    vals=', '.join(fmt(v) for v in flat)
    return f'static const {ctype} {name}[{len(flat)}] = {{{vals}}};\n'

h=[]
h.append('// Auto-generated by Mini Project 15. Frozen deployment package.\n#pragma once\n#include <stdint.h>\n')
h.append(f'#define MP15_INPUT_DIM {len(sel)}\n#define MP15_N_CLASSES {len(classes)}\n')
h.append(c_arr('MP15_FEATURE_INDEX', sel, 'int32_t', lambda v:str(int(v))))
h.append(c_arr('MP15_FEATURE_MEAN', mean39[sel], 'float', lambda v:f'{float(v):.9g}f'))
h.append(c_arr('MP15_FEATURE_SCALE', scale39[sel], 'float', lambda v:f'{float(v):.9g}f'))
h.append(f'static const float MP15_INPUT_Q_SCALE = {x_scale:.9g}f;\n')
h.append(c_arr('MP15_WEIGHT_Q', Wq, 'int8_t', lambda v:str(int(v))))
h.append(c_arr('MP15_WEIGHT_SCALE', W_scales, 'float', lambda v:f'{float(v):.9g}f'))
h.append(c_arr('MP15_BIAS', b, 'float', lambda v:f'{float(v):.9g}f'))
for i,c in enumerate(classes): h.append(f'// class {i}: {c}\n')
(firmware_dir/'mp15_classical_model_int8.h').write_text(''.join(h), encoding='utf-8')

# ---------- B. Tiny CNN float -> static INT8 PTQ ----------
class FloatTinyKWS(nn.Module):
    def __init__(self,n_classes):
        super().__init__()
        self.features=nn.Sequential(
            nn.Conv2d(1,8,3,padding=1,bias=False), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8,16,3,padding=1,bias=False), nn.BatchNorm2d(16), nn.ReLU(), nn.AdaptiveAvgPool2d((1,1)))
        self.classifier=nn.Linear(16,n_classes)
    def forward(self,x):
        return self.classifier(self.features(x).flatten(1))

class QuantTinyKWS(nn.Module):
    def __init__(self,n_classes):
        super().__init__()
        self.quant=torch.ao.quantization.QuantStub()
        self.conv1=nn.Conv2d(1,8,3,padding=1,bias=False); self.bn1=nn.BatchNorm2d(8); self.relu1=nn.ReLU(); self.pool=nn.MaxPool2d(2)
        self.conv2=nn.Conv2d(8,16,3,padding=1,bias=False); self.bn2=nn.BatchNorm2d(16); self.relu2=nn.ReLU(); self.gap=nn.AdaptiveAvgPool2d((1,1))
        self.fc=nn.Linear(16,n_classes); self.dequant=torch.ao.quantization.DeQuantStub()
    def forward(self,x):
        x=self.quant(x); x=self.relu1(self.bn1(self.conv1(x))); x=self.pool(x); x=self.relu2(self.bn2(self.conv2(x))); x=self.gap(x); x=x.flatten(1); x=self.fc(x); return self.dequant(x)

ckpt=torch.load(ROOT/dl_cfg['model_file'], map_location='cpu')
mu=float(ckpt['normalization']['mean']); sigma=float(ckpt['normalization']['std'])
float_net=FloatTinyKWS(len(classes)); float_net.load_state_dict(ckpt['state_dict']); float_net.eval()
qnet=QuantTinyKWS(len(classes))
mp={
'features.0.weight':'conv1.weight','features.1.weight':'bn1.weight','features.1.bias':'bn1.bias','features.1.running_mean':'bn1.running_mean','features.1.running_var':'bn1.running_va
r','features.1.num_batches_tracked':'bn1.num_batches_tracked',
'features.4.weight':'conv2.weight','features.5.weight':'bn2.weight','features.5.bias':'bn2.bias','features.5.running_mean':'bn2.running_mean','features.5.running_var':'bn2.running_va
r','features.5.num_batches_tracked':'bn2.num_batches_tracked','classifier.weight':'fc.weight','classifier.bias':'fc.bias'}
qnet.load_state_dict({mp[k]:v for k,v in ckpt['state_dict'].items()}, strict=True); qnet.eval()
torch.ao.quantization.fuse_modules(qnet, [['conv1','bn1','relu1'],['conv2','bn2','relu2']], inplace=True)
engine='fbgemm' if 'fbgemm' in torch.backends.quantized.supported_engines else torch.backends.quantized.supported_engines[0]
torch.backends.quantized.engine=engine
qnet.qconfig=torch.ao.quantization.get_default_qconfig(engine)
torch.ao.quantization.prepare(qnet, inplace=True)
# Calibration TRAINING ONLY.
with torch.no_grad():
    for sid in split['train_ids']:
        x=load_logmel(sid,mu,sigma)
        qnet(torch.from_numpy(x[None,None]).float())
torch.ao.quantization.convert(qnet, inplace=True); qnet.eval()

# Prediction agreement after quantization. This is equivalence checking, not retuning.
all_ids=list(split['train_ids'])+list(split['validation_ids'])+list(split['test_ids'])
float_cnn=[]; quant_cnn=[]
with torch.no_grad():
    for sid in all_ids:
        x=load_logmel(sid,mu,sigma); xt=torch.from_numpy(x[None,None]).float()
        float_cnn.append(classes[int(float_net(xt).argmax(1).item())])
        quant_cnn.append(classes[int(qnet(xt).argmax(1).item())])
float_cnn=np.asarray(float_cnn); quant_cnn=np.asarray(quant_cnn)
ntr=len(split['train_ids']); nv=len(split['validation_ids']); test_slice=slice(ntr+nv, len(all_ids))
cnn_agree_all=float(np.mean(float_cnn==quant_cnn)); cnn_agree_test=float(np.mean(float_cnn[test_slice]==quant_cnn[test_slice]))

# TorchScript host artifact. Note: this is not a TFLM flatbuffer.
example=torch.randn(1,1,TARGET_FRAMES,26)
scripted=torch.jit.trace(qnet, example)
qcnn_path=model_dir/'mp15_tiny_cnn_int8_torchscript.pt'
torch.jit.save(scripted, str(qcnn_path))

# ---------- Benchmark host reference ----------
ix0=np.where(train_mask)[0][0]; z0=Z8[ix0]
bench_svm_float=bench(lambda: svm.predict(z0.reshape(1,-1)), iters=300)
bench_svm_int8=bench(lambda: svm_int8_scores(z0), iters=300)
x0=load_logmel(split['train_ids'][0],mu,sigma); xt0=torch.from_numpy(x0[None,None]).float()
with torch.no_grad():
    bench_cnn_float=bench(lambda: float_net(xt0), iters=100)
    bench_cnn_int8=bench(lambda: qnet(xt0), iters=100)

# Payload estimates, not runtime/library overhead.
classical_payload_bytes=int(Wq.size + 4*len(W_scales) + 4*len(b) + 4*len(sel)*3 + 4)
dlmet=json.loads((ROOT/'results/metrics/mp12_deep_learning_metrics.json').read_text(encoding='utf-8'))
cnn_int8_weight_bytes=int(dlmet['estimated_int8_parameter_bytes'])

# Frozen-test performance from MP13. Selection policy: within PERF_TOL, smaller payload wins.
f1_class=float(mp13['classical']['macro_f1']); f1_cnn=float(mp13['tiny_cnn']['macro_f1'])
best_f1=max(f1_class,f1_cnn)
choices=[]
if f1_class>=best_f1-PERF_TOL: choices.append(('classical_svm_int8', classical_payload_bytes, bench_svm_int8['p95_ms']))
if f1_cnn>=best_f1-PERF_TOL: choices.append(('tiny_cnn_int8', cnn_int8_weight_bytes, bench_cnn_int8['p95_ms']))
selected=sorted(choices, key=lambda x:(x[1],x[2]))[0][0]

config={
```

```
    'source_status':'ready_for_mp15',
    'post_test_tuning_performed':False,
    'cnn_ptq_calibration_scope':'training_only',
    'svm_input_quant_calibration_scope':'training_only',
    'selection_uses_frozen_mp13_metrics':True,
    'selection_policy':f'candidate within {PERF_TOL:.2f} macro-F1 of best -> smaller deployment payload -> lower reference p95 latency',
    'selected_deployment_model':selected,
    'primary_target_profile':'MCU_first' if selected=='classical_svm_int8' else 'MPU_or_accelerated_MCU',
    'fallback_candidate':'tiny_cnn_int8' if selected=='classical_svm_int8' else 'classical_svm_int8',
    'realtime_contract':'config/realtime_architecture_config.json',
    'classical_package':'models/mp15_classical_int8.npz',
    'classical_c_header':'firmware/mp15_classical_model_int8.h',
    'tiny_cnn_host_quantized_package':'models/mp15_tiny_cnn_int8_torchscript.pt',
    'note':'TorchScript INT8 is used for host numerical/latency validation. MP16 ports the selected final model to the actual embedded runtime.'
}
save_json(ROOT/'config/deployment_config.json',config)

report={
    'frozen_test_macro_f1':{'classical':f1_class,'tiny_cnn':f1_cnn},
    'svm_int8':{
        'agreement_all_24_vs_float':svm_agree_all,
        'agreement_test_vs_float':svm_agree_test,
        'input_quant_scale':x_scale,
        'weight_scales':W_scales.tolist(),
        'payload_bytes_estimate':classical_payload_bytes,
        'host_reference_latency':bench_svm_int8,
        'float_host_reference_latency':bench_svm_float
    },
    'tiny_cnn_int8':{
        'calibration_scope':'training_only',
        'quantized_engine':engine,
        'agreement_all_24_vs_float':cnn_agree_all,
        'agreement_test_vs_float':cnn_agree_test,
        'weight_bytes_estimate':cnn_int8_weight_bytes,
        'torchscript_file_bytes':int(qcnn_path.stat().st_size),
        'host_reference_latency':bench_cnn_int8,
        'float_host_reference_latency':bench_cnn_float
    },
    'selection':{
        'performance_tolerance':PERF_TOL,
        'selected_model':selected,
        'reason':'Frozen-test performance is within tolerance; deployment payload is smaller and reference latency is lower.'
    },
    'device_measurement_note':'Host latency is not target-device latency. Actual latency, peak RAM, flash and power are measured during MP16 integration.',
    'post_test_tuning_performed':False
}
save_json(ROOT/'results/metrics/mp15_deployment_metrics.json',report)

# Manifest for MP16.
manifest={
    'selected_model':selected,
    'classes':classes,
    'sample_rate_hz':rt['sample_rate_hz'],
    'dma_block_samples':rt['dma_block_samples'],
    'feature_frame_samples':rt['feature_frame_samples'],
    'feature_hop_samples':rt['feature_hop_samples'],
    'inference_stride_frames':rt['inference_stride_frames'],
    'selected_feature_names':red['selected_feature_names'] if selected=='classical_svm_int8' else None,
    'files_required':[
        'config/project_config.json','config/filter_final_config.json','config/preprocessing_config.json','config/feature_config.json',
        'config/realtime_architecture_config.json','config/deployment_config.json','firmware/mp14_realtime_constants.h',
        'firmware/mp15_classical_model_int8.h' if selected=='classical_svm_int8' else 'models/mp15_tiny_cnn_int8_torchscript.pt'
    ]
}
save_json(ROOT/'deployment/mp15_manifest.json',manifest)

# Figures
figdir=ROOT/'results/figures'; figdir.mkdir(parents=True,exist_ok=True)
plt.figure(figsize=(7.2,4.5));
names=['SVM float\nserialized','SVM INT8\npayload','CNN FP32\nweights','CNN INT8\nweights']
vals=[int(mp13['resource_proxy']['classical']['serialized_bytes']), classical_payload_bytes, int(dlmet['float32_parameter_bytes']), cnn_int8_weight_bytes]
plt.bar(names,vals); plt.ylabel('Byte'); plt.title('MP15 - Ukuran model/payload deployment'); plt.grid(axis='y',alpha=.2); plt.tight_layout();
plt.savefig(figdir/'mp15_model_size.png',dpi=170); plt.close()

plt.figure(figsize=(7.2,4.5));
names=['SVM float','SVM INT8 ref','CNN float','CNN INT8']
vals=[bench_svm_float['p95_ms'],bench_svm_int8['p95_ms'],bench_cnn_float['p95_ms'],bench_cnn_int8['p95_ms']]
plt.bar(names,vals); plt.ylabel('p95 latency host (ms)'); plt.title('MP15 - Benchmark referensi host'); plt.grid(axis='y',alpha=.2); plt.tight_layout();
plt.savefig(figdir/'mp15_latency.png',dpi=170); plt.close()

plt.figure(figsize=(7.2,4.5));
plt.bar(['SVM all','SVM test','CNN all','CNN test'],[svm_agree_all,svm_agree_test,cnn_agree_all,cnn_agree_test]); plt.ylim(0,1.05); plt.ylabel('Prediction agreement'); plt.title('MP15 -
Kesetaraan prediksi sebelum/sesudah kuantisasi'); plt.grid(axis='y',alpha=.2); plt.tight_layout(); plt.savefig(figdir/'mp15_equivalence.png',dpi=170); plt.close()

status['status']='ready_for_mp16'; status['last_completed_mini_project']=15
status['mp15']={
    'deployment_config':'config/deployment_config.json',
    'deployment_metrics':'results/metrics/mp15_deployment_metrics.json',
    'deployment_manifest':'deployment/mp15_manifest.json',
    'selected_model':selected,
    'classical_package':'models/mp15_classical_int8.npz',
    'classical_header':'firmware/mp15_classical_model_int8.h',
    'tiny_cnn_quantized_host_artifact':'models/mp15_tiny_cnn_int8_torchscript.pt',
    'post_test_tuning_performed':False,
    'device_benchmark_deferred_to_mp16':True
}
save_json(status_path,status)

print('Mini Project 15 selesai.')
print('SVM agreement all/test:',svm_agree_all,svm_agree_test)
print('CNN agreement all/test:',cnn_agree_all,cnn_agree_test)
```

```
print('Selected deployment model:',selected)
print('Status:',status['status'])
```

Kriteria Mini Project 15

Pemeriksaan	Kriteria lulus
Status awal	ready_for_mp15
Calibration CNN	training-only
Calibration SVM input	training-only
Post-test tuning	False
SVM agreement	dicatat, ideal 1,00 pada running project
CNN agreement	dicatat, ideal 1,00 pada running project
C header	mp15_classical_model_int8.h tersedia
Deployment manifest	mp15_manifest.json tersedia
Model selection	tercatat dengan policy
Status akhir	ready_for_mp16

Tabel 15.12. Checklist kelulusan Mini Project 15.

Artefak yang wajib dibawa ke Mini Project 16

Jangan menghapus config/deployment_config.json, deployment/mp15_manifest.json, firmware/mp14_realtime_constants.h, firmware/mp15_classical_model_int8.h, models/mp15_classical_int8.npz, models/mp15_tiny_cnn_int8_torchscript.pt, seluruh konfigurasi filter/preprocessing/feature, dan project_status.json. MP16 menggunakan artefak ini untuk integrasi sistem end-to-end pada Raspberry Pi atau ESP32 sesuai platform yang tersedia.

Referensi

1. Warden, P., & Situnayake, D. (2019). TinyML. O'Reilly Media.

2. Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2020). Efficient Processing of Deep Neural Networks. Morgan & Claypool.

3. Jacob, B., Kligys, S., Chen, B., et al. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. Proceedings of CVPR.

4. Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. ICLR.

5. Lai, L., Suda, N., & Chandra, V. (2018). CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs. arXiv:1801.06601.

6. Wang, E., Davis, J. J., Zhao, R., et al. (2019). Deep Neural Network Approximation for Custom Hardware: Where We've Been, Where We're Going. ACM Computing Surveys.

7. ARM. CMSIS-NN and CMSIS-DSP documentation. Reference for optimized embedded kernels.

8. TensorFlow Lite for Microcontrollers documentation. Reference for integer inference, operator resolver, and tensor arena concepts.